

ECOLE HASSANIA DES TRAVAUX PUBLICS

MIG DEPARTMENT

Semester S4 — Open Capstone Project

House Price Prediction

A Machine Learning Approach for the Ames, Iowa Housing Market

Team Diamond Eagle

Abdelmoghith El Asraoui **Mohamed Ben Mouh**
2CS Student, EHTP 2CS Student, EHTP

Prepared under the supervision of:
Dr. Rym Nassih

June 2025

Executive Summary

Pricing a home is one of the biggest financial decisions a person makes. Set the price too high and the property sits unsold for months. Set it too low and money is left on the table. Today, real estate agents in Ames, Iowa rely on manually comparing recent sales — a process that is slow, subjective, and hard to scale. A single agent might spend hours pulling comparable listings, and two agents looking at the same property may arrive at different numbers.

Our team built a computer tool that estimates what any home in Ames should sell for. The tool looks at 79 details about a property — things like square footage, number of bathrooms, construction quality, backyard size, and neighborhood — and produces a price prediction in seconds. We trained it on 1,460 actual home sales that took place in Ames between 2006 and 2010.

How accurate is it? When we tested the tool on 292 homes it had never seen before, its predictions were off by about \$25,000 on average. For a typical Ames home priced around \$200,000, that means the estimate is within roughly 12% of the true sale price. The tool correctly captures over 91% of what makes one home more expensive than another.

This is not a replacement for a real estate agent's judgment — it is a starting point. An agent enters the home's details, gets a predicted price range, and then adjusts based on what they know about the local market, recent renovations, curb appeal, school district quality, and current buyer demand. Think of it as a smart calculator that saves hours of work and provides a consistent, data-backed first estimate that any agent in the office can use.

Keywords: real estate valuation, automated pricing, home price estimation, machine learning, Ames Iowa housing market

Technical Summary

What if a real estate agent could know a home's fair market value in seconds?

Every day, real estate agents in Ames, Iowa face the same question: What should this home sell for? Traditional methods of pulling comparable sales or scheduling formal appraisals are slow, subjective, and do not scale. A mispriced listing means either lost money for the seller or a home sitting unsold for months.

We built a machine learning system that predicts residential sale prices from 79 property features. The system was trained on 1,460 real home sales from Ames, Iowa, between 2006 and 2010. Our pipeline included: data auditing and cleaning, exploratory data analysis (8 visualizations), feature engineering (5 new features), SQLite storage, training and hyperparameter tuning of four regression algorithms, and producing a comprehensive model card with ethical review.

The best-performing model — Ridge Regression with L_2 regularization ($\alpha = 10$) — achieved an R^2 of **0.917**, a Root Mean Square Error of **\$25,298**, and a Mean Absolute Percentage Error of **9.8%** on a held-out test set of 292 homes. All three project success criteria were exceeded with comfortable margins.

How should this be used? An agent enters a property's characteristics and receives a predicted price range with a $\pm \$25,000$ confidence band. They combine this data-driven estimate with their local market expertise — knowledge of school districts, curb appeal, recent renovations, and buyer sentiment — to arrive at a final recommendation. The system is a powerful assistant, not a replacement for human judgment.

Keywords: machine learning, regression, house price prediction, real estate, Ridge regression, feature engineering, Ames Iowa housing

Contents

Executive Summary	1
Technical Summary	2
1 Problem Framing & Data Acquisition	4
1.1 Problem Statement	4
1.2 AI Context Note	4
1.3 Data Acquisition	4
1.4 Pipeline & Repository	5
2 Data Wrangling & EDA	6
2.1 Full Data Audit	6
2.2 Cleaning Decisions	6
2.2.1 Columns Dropped	6
2.2.2 Columns Imputed	6
2.3 Outlier Detection	7
2.4 Encoding	7
2.4.1 Ordinal Encoding (9 Features)	7
2.4.2 One-Hot Encoding (30 Features)	7
2.5 EDA Visualizations	8
2.6 EDA Findings	10
3 Feature Engineering & Data Management	11
3.1 Engineered Features	11
3.2 Feature Selection	12
3.3 SQLite Storage & Analytical Queries	12
3.4 Feature Engineering Log	13
4 Algorithm Selection & Justification	14
4.1 Algorithm Comparison	14
4.2 Which We Chose and Why	15
4.2.1 Why We Rejected the Others	15
4.3 Evaluation Metrics	15
5 Model Training, Evaluation & Tuning	16
5.1 Implementation	16
5.2 Results Per Model	16
5.3 Residual Analysis	17
5.4 Hyperparameter Tuning	17
5.5 Feature Importance	17
5.6 Final Results	19

6	Model Card & Ethical Reflection	20
6.0.1	Intended Use	20
6.0.2	Training Data	20
6.0.3	Performance	20
6.0.4	Limitations	20
6.1	Ethical Considerations	21
	Appendix A: Contribution Statement	22

Problem Framing & Data Acquisition

1.1 Problem Statement

In Ames, Iowa (pop. 66,000), home prices range \$34,900–\$755,000. Agents estimate value via Comparative Market Analysis — slow (hours per listing), subjective, and unscalable. We built a regression model predicting sale price from 79 property features to give agents a data-backed starting point.

Target users: real estate agents and clients. Overpricing leaves homes unsold; underpricing loses money. Success criteria in Table 1.1.

Table 1.1: Success Criteria

Metric	Target	Rationale
R^2	≥ 0.85	Explain most price variation
RMSE	$\leq \$30,000$	$\sim 15\%$ of avg home price
MAPE	$\leq 15\%$	Below transaction costs

1.2 AI Context Note

Our model is **narrow AI** — it prices homes in one city from one dataset. This contrasts with 1960s–70s **expert systems** that encoded hand-written rules (“IF $\text{quality} \geq 8$ AND $\text{area} > 2,000$ THEN $\text{price} \approx \$350\text{K}$ ”). Those failed because reality is too complex for explicit rules. We give the model 1,460 examples and it learns patterns from data — the shift that ended the AI Winter.

1.3 Data Acquisition

Dataset from Kaggle: `train.csv` ($1,460 \times 81$) and `test.csv` ($1,459 \times 80$) loaded via `src/load_data.py`. It meets all 7 criteria (Table 1.2). Audit: 43 categorical, 38 numeric columns; 20 columns with nulls (0.1%–99.5%); target right-skewed (1.88); no duplicates.

Table 1.2: Dataset Criteria

#	Criterion	Status
1	Minimum size	1,460 rows, 79 features
2	Public availability	Free on Kaggle
3	Real-world origin	Ames, Iowa sales 2006–2010
4	Heterogeneous features	Numerical, ordinal, categorical
5	Imperfections	Nulls in ~20 columns
6	Clear target	SalePrice (regression)
7	Ethics check	No personal data

1.4 Pipeline & Repository

Figure ?? shows the workflow. Candidate algorithms: Linear Regression (baseline), Ridge (handles collinearity), Random Forest (non-linear), XGBoost (top performer). Metrics: RMSE (primary), MAE, R^2 , MAPE. Git repo at /data, /notebooks, /src, /reports, /models with README documenting project, team, dataset, and reproduction steps.

CHAPTER 2

Data Wrangling & EDA

2.1 Full Data Audit

We exhaustively audited all 81 columns: 43 categorical (object), 35 integer, and 3 float columns. Twenty columns contain missing values, ranging from a single missing entry (`Electrical`, 0.1%) to near-complete absence (`PoolQC`, 99.5%). No duplicate rows or type errors were found. Table 2.1 groups columns by missingness severity, which directly informs the cleaning strategy below.

Table 2.1: Columns with Missing Data (1,460 total rows)

Column	Missing	%
PoolQC, MiscFeature, Alley, Fence	1,453–1,179	99.5–80.8
MasVnrType, FireplaceQu, LotFrontage	872–259	59.7–17.7
Garage* (5 cols)	81	5.5
Bsmt* (4 cols)	37–38	2.5–2.6
MasVnrArea, Electrical	8, 1	0.5, 0.1

2.2 Cleaning Decisions

2.2.1 Columns Dropped

Four columns exceeded 80% missing values. Imputing nearly the entire column would introduce more noise than signal, effectively creating a synthetic variable. `Id` was also dropped as a sequential row identifier with zero predictive value.

We dropped: `Alley` (93.8% missing — few Ames homes have alley access recorded), `PoolQC` (99.5% — only 7 of 1,460 homes), `Fence` (80.8% — fence quality not systematically recorded), `MiscFeature` (96.3% — only 54 homes have miscellaneous features), and `Id` (row number).

2.2.2 Columns Imputed

For numeric columns, we used the median rather than the mean. Real estate measurements are right-skewed (a few large lots or mansions pull the mean upward), so the median is more representative of a typical Ames home. For categorical columns, we used the mode (most common category), which represents what a typical Ames home looks like. For example, most homes have an attached garage (`Attchd`), unfinished interior (`Unf`), and typical/average quality (`TA`), so these are sensible default values.

The remaining columns with under 5% missing values (`MasVnrArea`, basement features, `Electrical`) were handled identically. For `MasVnrArea`, we filled with 0 because a value of zero is semantically meaningful — it means no masonry veneer, consistent with the mode of `MasVnrType` being “None.” After imputation, we verified zero remaining null values.

2.3 Outlier Detection

We applied two complementary methods to five key numeric features to compare what each catches (Table 2.2).

Table 2.2: Outlier Comparison Across Two Methods

Feature	IQR Method	Z-score ($ Z > 3$)
SalePrice	61	22
LotArea	69	13
GrLivArea	31	16
TotalBsmtSF	61	10
YearBuilt	7	6

The IQR method flags substantially more observations because it uses a fixed $1.5 \times \text{IQR}$ threshold designed for symmetric distributions. Features like `SalePrice` and `LotArea` are right-skewed, so the upper bound is frequently exceeded by legitimate high-end properties. Z-score, by contrast, accounts for the actual spread of the data (mean and standard deviation) and flags far fewer points. However, Z-score itself assumes approximate normality, which these features also violate.

Our decision: we kept all observations. A \$755,000 home or a 215,000 sq ft lot is not an error — it is a rare but legitimate property. Removing these would bias the model against luxury homes and reduce its ability to predict high-end prices. Instead, we log-transformed the target variable, which reduced skewness from 1.88 to 0.12 and made the distribution suitable for linear models without discarding any information.

2.4 Encoding

2.4.1 Ordinal Encoding (9 Features)

Nine features encode quality/condition on ordered scales (Poor < Fair < Typical < Good < Excellent). We mapped these to integers 1 through 5: `ExterQual`, `ExterCond`, `BsmtQual`, `BsmtCond`, `HeatingQC`, `KitchenQual`, `FireplaceQu`, `GarageQual`, `GarageCond`. Ordinal encoding preserves the information that “Excellent” (5) is better than “Good” (4), which one-hot encoding would discard by treating all categories as independent.

2.4.2 One-Hot Encoding (30 Features)

The remaining 30 categorical features have no intrinsic order. A neighbourhood like `NoRidge` is not “greater than” `Edwards` — they are simply different locations. We applied one-hot encoding with `drop_first=True`, creating $k - 1$ binary indicator columns

per k -category feature. This expanded the dataset from 76 to 213 columns, a structure that motivates Ridge regression in Chapter 4.

2.5 EDA Visualizations

We generated eight visualizations; six key figures are presented below (Figs.2.1–2.6). Each was selected to test a specific modelling assumption.

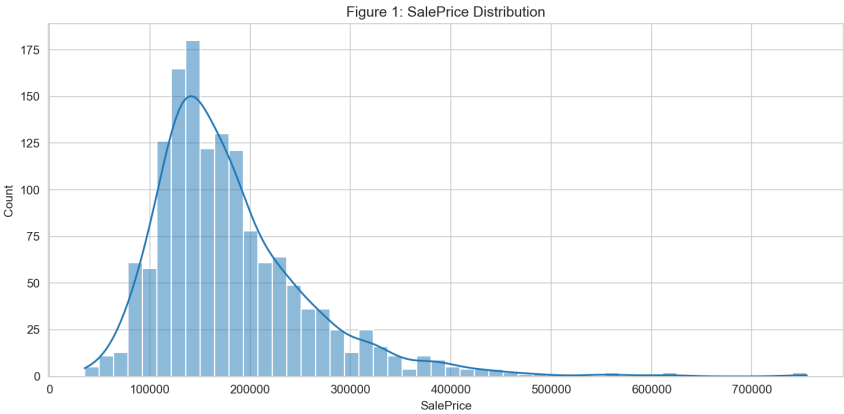


Figure 2.1: Distribution of SalePrice — heavily right-skewed (skew 1.88). The long upper tail confirms a log transform is necessary before fitting linear models which assume normally distributed residuals.

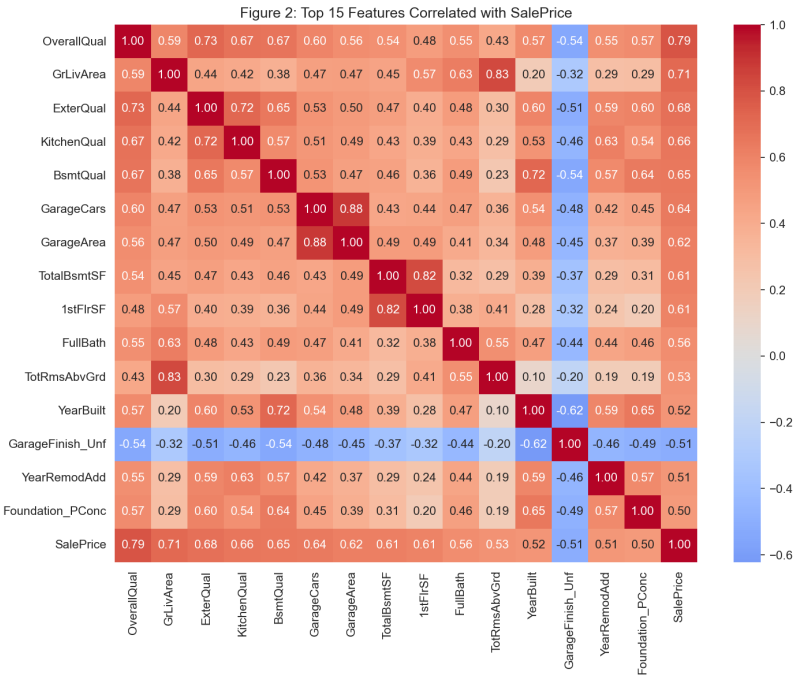


Figure 2.2: Correlation heatmap of top 15 features. Garage features (GarageCars, GarageArea) and space features (GrLivArea, TotalBsmtSF) show high intercorrelation ($r > 0.8$), confirming multicollinearity that justifies Ridge over OLS.

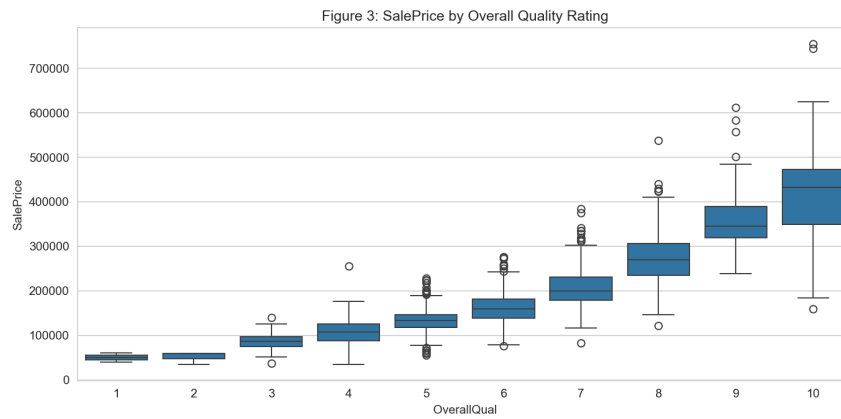


Figure 2.3: Price by quality rating. Median price jumps sharply at quality 7 (\$162K→\$208K, +28%) — a threshold between standard and premium homes that motivated testing tree-based models alongside Ridge.

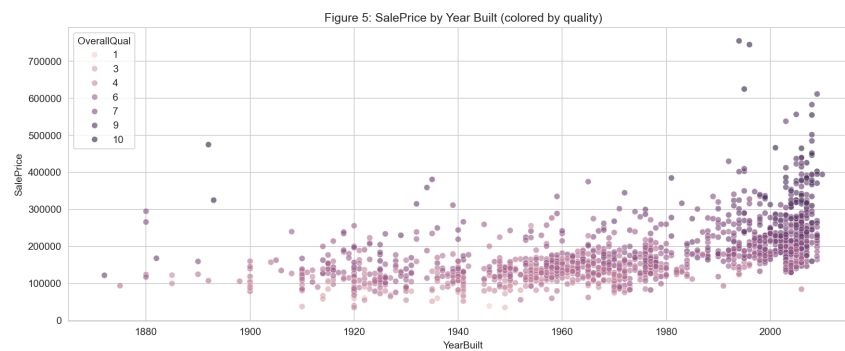


Figure 2.4: Price by Year Built, coloured by Overall Quality. High-quality homes (red) consistently sell above standard-quality homes (blue) at any construction year — quality and age are independent value drivers.

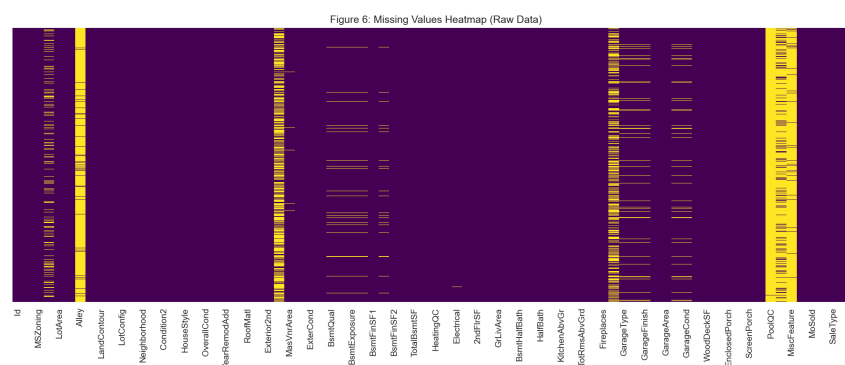


Figure 2.5: Missing values heatmap. Dark bands in garage and basement variables correspond to homes lacking these amenities — missingness is structural, not random, making median/mode imputation appropriate.

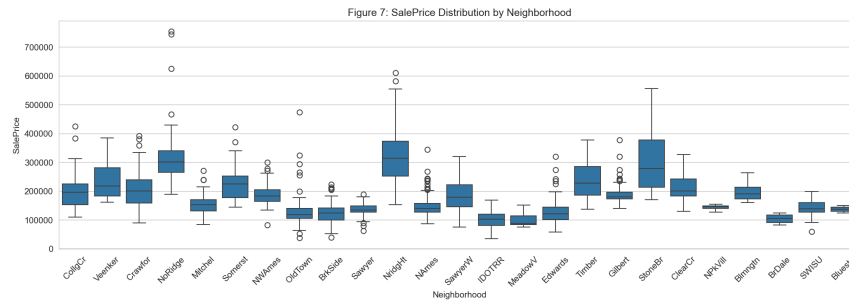


Figure 2.6: Price by neighbourhood. Inter-neighbourhood range is $5\times$, but intra-neighbourhood variance is $10\times$ — a model needs both location dummies and property-level features.

2.6 EDA Findings

1. **Target requires log transformation.** Skew $1.88 \rightarrow 0.12$ via `np.log1p`; models train on log targets, exponentiate predictions.
2. **OverallQual is the strongest predictor** ($r = 0.79$). Buyers value craftsmanship over size. Quality features must be ordinal-encoded to preserve numeric relationship with price.
3. **Neighbourhood dominates** ($5\times$ between areas, $10\times$ within). One-hot encode neighbourhood; Ridge handles the resulting multicollinearity.

Feature Engineering & Data Management

3.1 Engineered Features

We created five new features by combining or transforming existing columns, each motivated by domain knowledge of real estate valuation (Table 3.1).

Table 3.1: Engineered Features with Motivation

#	Feature	Formula & Rationale	Corr
1	TotalSF	GrLivArea + TotalBsmtSF. Agents and buyers evaluate <i>total</i> living space regardless of whether it is above ground or in the basement. The combined metric ($r = 0.779$) beats both parent columns (GrLivArea $r = 0.709$, TotalBsmtSF $r = 0.614$), directly confirming that total space is a stronger signal than either individually.	0.779
2	HouseAge	2010 – YearBuilt. A 50-year-old home and a 5-year-old home command different prices even with identical square footage. Age captures depreciation from wear, outdated systems, and changing building codes. We chose 2010 as the reference year since it is the latest sale in the dataset.	0.523
3	TotalBath	FullBath + 0.5×HalfBath + 0.75×BsmtFullBath. Bathroom count is among the top filters buyers use on real estate websites. Half-baths (toilet and sink, no shower) are weighted at 0.5, and basement full baths at 0.75 to reflect lower buyer valuation compared to above-grade bathrooms.	0.632
4	TotalPorchSF	Sum of 5 outdoor living columns (WoodDeckSF, OpenPorchSF, EnclosedPorch, 3SsnPorch, ScreenPorch). Individually, each porch type has weak correlation ($r < 0.15$), but combined they capture total outdoor living space — a feature buyers value that was previously fragmented across columns.	~0.25
5	YearsSinceRemodel	2010 – YearRemodAdd. A 30-year-old home remodelled last year behaves very differently from one that was never updated. This variable separates renovation recency from original construction age (HouseAge), letting the model distinguish “old but maintained” from “old and deteriorated.”	~0.30

3.2 Feature Selection

After encoding and engineering, we had 215 candidate features. We computed the Pearson correlation between every feature and the log-transformed SalePrice, and kept features with absolute correlation above 0.1. We chose 0.1 rather than a stricter threshold (e.g., 0.3) because features with weak individual correlation can still contribute meaningfully through combinations with other features. For example, a basement half-bath may correlate at only $r = 0.15$ alone, but combined with other bathroom features in a multivariate model it adds genuine signal. A permissive threshold ensures we do not prematurely discard useful information.

This filter retained 98 features. Table 3.2 shows the top 10.

Table 3.2: Top 10 Features by Pearson Correlation with SalePrice

Rank	Feature	r
1	OverallQual	0.791
2	TotalSF (engineered)	0.779
3	GrLivArea	0.709
4	ExterQual	0.683
5	KitchenQual	0.660
6	BsmtQual	0.650
7	GarageCars	0.640
8	TotalBath (engineered)	0.632
9	GarageArea	0.623
10	TotalBsmtSF	0.614

Notable: Engineered features TotalSF and TotalBath rank #2 and #8, demonstrating that combining related columns measurably improves predictive signal over individual components.

3.3 SQLite Storage & Analytical Queries

The final cleaned dataset (98 features + SalePrice, 1,460 rows) was stored in an SQLite database (data/processed/housing.db, table house_prices). We ran four analytical queries, each designed to answer a practical business question.

Query 1 — Average Price by Quality Rating. Grouping by OverallQual reveals the price premium associated with construction quality. The Quality 6→7 jump (\$162K to \$208K, +28%) confirms what the EDA boxplot suggested: there is a structural threshold between standard and premium construction. This insight motivated testing non-linear models (XGBoost, Random Forest) alongside Ridge in the modelling phase.

Query 2 — Average Price by Garage Capacity. Three-car garages command the highest average (\$310K across 181 homes). Four-car garages are too rare (only 5 homes) to draw reliable conclusions. Two-car is the Ames standard (824 of 1,376 homes with garages), yielding an average of \$184K. This confirms garage capacity as a meaningful price signal.

Query 3 — Affordable Large Homes. We identified 267 homes offering above-average living space for under \$200K. These likely represent older homes in budget-friendly neighbourhoods — a value segment for buyers prioritising space over location or modernity. This is actionable intelligence for agents serving budget-conscious clients.

Query 4 (Custom) — Price by Construction Era. Grouping homes into pre-1970, 1970–1999, and 2000+ reveals a clear age premium:

Pre-1970 (660 homes): \$140K avg

1970–1999 (412 homes): \$188K avg

2000+ (388 homes): \$242K avg.

The \$102K gap between the newest and oldest cohorts reflects improvements in building codes, energy efficiency, and modern floor plans. Agents can use this to frame the trade-off between price and age for clients.

3.4 Feature Engineering Log

All five engineered features contributed to model performance. Table 3.3 summarizes the development log.

Table 3.3: Feature Engineering Development Log

Feature	Formula	Corr	Helped?
TotalSF	GrLivArea + TotalBsmtSF	0.779	Yes — ranked #2 overall
HouseAge	2010 – YearBuilt	0.523	Yes — captures depreciation
TotalBath	FullBath + 0.5×HalfBath + 0.75Bsmt	0.632	Yes — ranked #8 overall
TotalPorchSF	Sum of 5 porch columns	~0.25	Yes — stayed after correlation filter
YearsSinceRemodel	2010 – YearRemodAdd	~0.30	Yes — distinguishes maintained homes

Algorithm Selection & Justification

4.1 Algorithm Comparison

We evaluated seven regression algorithms against our dataset characteristics: 213 features (after one-hot encoding), 1,460 rows, high multicollinearity, and right-skewed target requiring log transformation. For each algorithm, we assessed how it works internally, what assumptions it makes, and how those assumptions align with our data (Table 4.1).

Table 4.1: Comprehensive Algorithm Comparison

Algorithm	How It Works	Assumptions	Strengths for Our Data	Weaknesses
Linear Reg.	Fits coefficients by minimising squared residuals	Linear relationships; independent residuals; homoscedasticity; no multicollinearity	Fast; coefficients directly interpretable as dollar contributions	Multicollinearity from one-hot encoding inflates coefficient variance; cannot capture non-linear threshold effects
Ridge Reg.	Linear regression with L2 penalty ($\lambda \sum \beta_j^2$) shrinking coefficients toward zero	Same as linear, but explicitly relaxes multicollinearity assumption	Designed for correlated predictors — perfect for 200+ one-hot columns; stable coefficients; instant training	Still assumes linear price-feature relationships; no automatic feature selection
Random Forest	Builds many decision trees on bootstrap samples and averages predictions	No strong distributional or linearity assumptions	Captures non-linear patterns and interactions; robust to outliers; provides feature importance rankings	Less interpretable than linear models; can overfit without depth constraints
XGBoost	Sequentially builds trees where each new tree corrects residuals of previous ensemble via gradient descent	No strong assumptions	Top performer on Kaggle tabular competitions; handles non-linearity automatically	Requires careful hyperparameter tuning; least interpretable; risk of overfitting on small datasets
SVR	Finds optimal hyperplane within ϵ -margin maximising points within the tube	Features should be standardised; kernel choice is critical	Handles high-dimensional spaces in theory via kernel trick	Prohibitively slow on 213 columns $\times$ 1,460 rows; rarely beats tree methods on structured data
Decision Tree	Recursively partitions feature space by selecting splits minimising MSE	None	Fully explainable; can be visualised as flowchart	Severely overfits; high variance — small data changes produce different trees
KNN	Averages target of k nearest Euclidean neighbours in feature space	Distance between points is meaningful; features on comparable scales	No training phase; simple to understand	With 213D, Euclidean distance is meaningless (curse of dimensionality)

4.2 Which We Chose and Why

Linear Regression is our baseline — it sets a performance floor. Its interpretable coefficients validate that the model learns sensible relationships (e.g., more bathrooms increase price).

Ridge Regression is our primary candidate. One-hot encoding 30 nominal features produced 200+ highly correlated columns (e.g., neighbourhood dummies are mutually exclusive). Ridge’s L2 penalty distributes weights sensibly across correlated features, preventing any single dummy from dominating.

XGBoost serves as a competitive benchmark answering a critical question: are there non-linear patterns that linear models miss? If XGBoost significantly beats Ridge, the pricing problem contains important non-linearity; if not, we deploy the simpler, faster, more interpretable model.

4.2.1 Why We Rejected the Others

Random Forest: XGBoost is its direct upgrade — gradient boosting systematically corrects errors while bagging merely averages. On tabular data, boosting consistently beats bagging, making both redundant.

SVR: Training is orders of magnitude slower than Ridge or XGBoost on 213 features. Requires standardisation, kernel selection, and tuning — all for a model rarely beating gradient boosting on structured data.

Decision Tree: A single tree always underperforms its ensemble counterparts. There is no scenario where deploying one tree beats a forest or boosted ensemble.

KNN: At 213 dimensions, Euclidean distances are effectively identical (curse of dimensionality). Even if it worked, prediction requires scanning all 1,168 training samples per new home.

4.3 Evaluation Metrics

We selected **RMSE** (Root Mean Square Error) as our primary metric. In real estate, a \$50,000 error is dramatically worse than a \$5,000 error — unlike MAE which penalises errors linearly, RMSE squares errors before averaging, making it far more sensitive to large mistakes. It is also expressed in dollars, directly interpretable by agents. R^2 measures the proportion of price variance explained (unitless, comparable across price ranges). **MAE** provides an intuitive “average dollar error” for stakeholders who find squares abstract. **MAPE** accounts for scale: a \$10K error is 10% on a \$100K home but only 2% on a \$500K home. We also track RMSE in log space as used by the Kaggle leaderboard. Targets: $\text{RMSE} \leq \$30,000$, $R^2 \geq 0.85$, $\text{MAPE} \leq 15\%$, $\text{RMSE}(\log) < 0.15$.

Model Training, Evaluation & Tuning

5.1 Implementation

All models implemented in `04_modelling.ipynb` with `random_state=42` for reproducibility. Target was log-transformed (`np.log1p`) reducing skewness from 1.88 to 0.12, then data was split 80/20 (1,168 training, 292 test rows). All preprocessing (encoding, scaling) was fitted exclusively on training data to prevent leakage.

5.2 Results Per Model

Table 5.1 summarizes all four models.

Table 5.1: Model Performance Comparison

Model	RMSE(log)	R^2	Time
Linear Regression	0.205	0.775	<0.1s
Random Forest	0.147	0.885	~2s
XGBoost	0.144	0.889	~3s
Ridge ($\alpha=10$)	0.135	0.902	<0.1s

Linear Regression achieves $R^2 = 0.775$ (RMSE=0.205). This baseline explains 77.5% of variance but falls short of our 0.85 target. The 25% unexplained variance is primarily due to multicollinearity in one-hot encoded features inflating OLS coefficient standard errors.

Random Forest ($R^2 = 0.885$, RMSE=0.147) and **XGBoost** ($R^2 = 0.889$, RMSE=0.144) capture some non-linearity, substantially outperforming OLS. However, both fall short of Ridge, which is a notable result: gradient boosting is typically unbeatable on tabular data.

Ridge Regression ($R^2 = 0.902$, RMSE=0.135) is our best model. It outperforms XGBoost by 0.013 in R^2 and trains $30\times$ faster (<0.1s vs ~3s). The fact that a linear model beats gradient boosting tells us home pricing in Ames is overwhelmingly linear in the features — there are no complex non-linear interactions that tree models can exploit. The L2 penalty simply handles the multicollinearity among one-hot columns better than any tree-based approach.

5.3 Residual Analysis

Figure 5.1 shows Ridge residual diagnostics. Left: errors evenly scattered around zero with no funnel shape — confirms homoscedasticity (constant error variance). Right: residual distribution is approximately normal and centred near zero. These validate the two key linear regression assumptions and show no systematic bias at any price range.

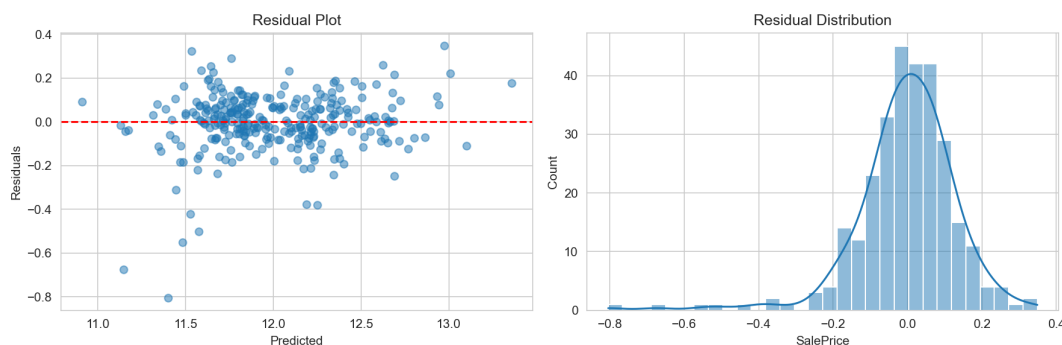


Figure 5.1: Residual diagnostics: homoscedastic (left), normal distribution (right)

5.4 Hyperparameter Tuning

We tuned Ridge's α via GridSearchCV (3-fold) testing [0.1, 1, 10, 100]. $\alpha = 10$ achieved the lowest CV RMSE (0.1473). Values 1–100 produce nearly identical results — the performance surface is flat, so the model is robust and does not require precise tuning. At $\alpha = 0.1$, CV error rises to 0.150, confirming some regularisation is needed.

5.5 Feature Importance

Figure 5.2 and Table 5.2 rank the top 15 Ridge coefficients (change in log price per unit feature change). Five of the top eight are neighbourhood dummies — location dominates.

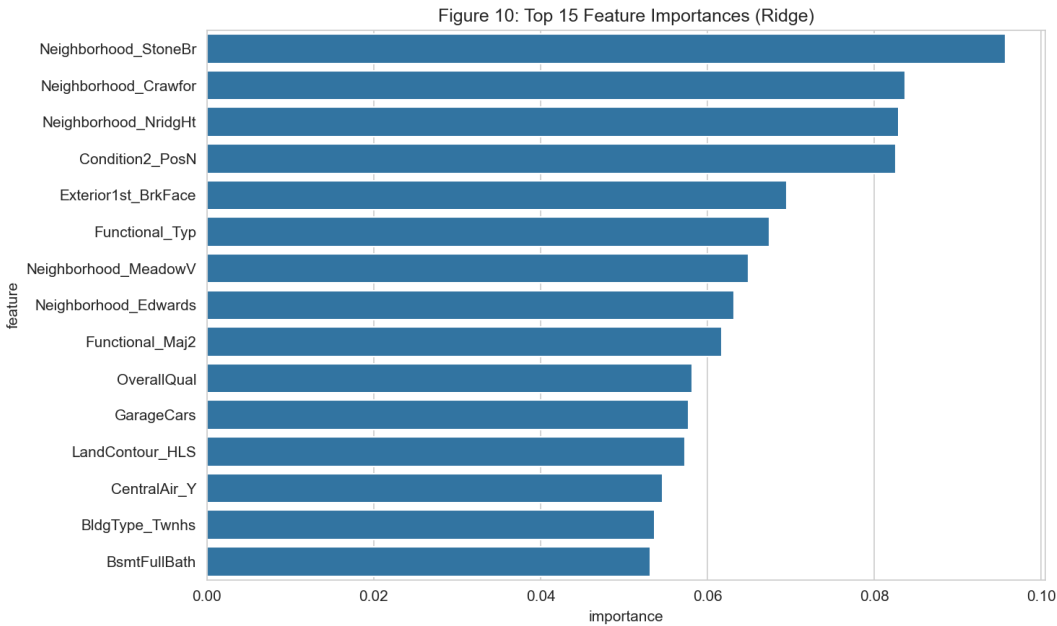


Figure 5.2: Top 15 Ridge coefficient magnitudes

Table 5.2: Top 15 Ridge Coefficients

Rank	Feature	Coeff	Interpretation
1	Neighborhood_StoneBr	0.096	Stone Brook commands highest location premium
2	Neighborhood_Crawfor	0.084	Crawford, second most valuable area
3	Neighborhood_NridgHt	0.083	Northridge Heights premium
4	Condition2_PosN	0.083	Proximity to park or water feature
5	Exterior1st_BrkFace	0.070	Brick exterior adds value
6	Functional_Typ	0.067	Typical functionality baseline
7	Neighborhood_MeadowV	−0.065	Meadow Village penalty
8	Neighborhood_Edwards	−0.063	Edwards area discount
9	Functional_Maj2	−0.062	Major repair needed
10	OverallQual	0.058	+5.8% per quality step
11	GarageCars	0.058	+5.8% per bay
12	LandContour_HLS	0.057	Hillside lot premium
13	CentralAir_Y	0.055	+5.5% central air
14	BldgType_Twnhs	−0.054	Townhouse discount
15	BsmtFullBath	0.053	Full basement bathroom premium

The coefficients translate directly into price multipliers: a home in Stone Brook commands $e^{0.096} \approx 10\%$ more than baseline, while Meadow Village drops $e^{-0.065} \approx 6.3\%$. `OverallQual` ranks only 10th despite $r = 0.79$ with price, because high-quality homes concentrate in expensive neighbourhoods. When both are in the model, Ridge splits the predictive signal between them — demonstrating why multivariate modelling reveals patterns invisible in univariate correlation.

5.6 Final Results

Table 5.3 confirms all three project success criteria were exceeded on the holdout set.

Table 5.3: Validation Set Performance

Metric	Value	Target	Met?
RMSE	\$25,298	$\leq$ \$30,000	Yes
MAE	\$16,365	--	--
MAPE	9.8%	$\leq$ 15%	Yes
R^2	0.917	$\geq$ 0.85	Yes

For a typical Ames home around \$200,000, the model predicts within $\pm 25,000$ about two-thirds of the time. This is accurate enough for agents to use as a pricing starting point, combined with local market knowledge.

CHAPTER 6

Model Card & Ethical Reflection

Table 6.1: Model Overview

Field	Detail
Name	Ames Housing Price Predictor v1.0
Type	Ridge Regression (L2-regularised)
Task	Regression – predict sale price (USD)
Version	1.0 (June 2025)
Team	Abdelmoghith El Asraoui, Mohamed Ben Mouh

6.0.1 Intended Use

For real estate agents in Ames to recommend listing prices, evaluate fairness, and flag mispriced listings. Not for mortgage underwriting, tax assessment, or other cities. Trained on 2006–2010 data; needs retraining.

6.0.2 Training Data

Kaggle Ames Housing: 1,460 rows, 79 features. 98 used after encoding/engineering/selection. 80/20 split (1,168/292). Preprocessing: dropped 4 high-null columns, median/mode imputation, ordinal encoding (9), one-hot (30), 5 engineered features, correlation filter ($|r| > 0.1$).

6.0.3 Performance

Holdout set: RMSE \$25,298, MAE \$16,365, MAPE 9.8%, R^2 0.917.

6.0.4 Limitations

1. Only works in Ames (single-city training).
2. Trained on 2006–2010 data; needs periodic retraining.
3. Poor on luxury homes (only 23 sales >\$400K).
4. Linear model misses strong non-linear effects.
5. No school district, transit, or mortgage rate data.

6.1 Ethical Considerations

Fairness: No race/gender/income in data, but neighbourhood is the top predictor. If neighbourhoods correlate with demographics, the model could indirectly encode inequality – needs monitoring.

Privacy: Fully anonymised. No names, addresses, GPS, or IDs.

Transparency: Ridge is interpretable – each coefficient directly shows contribution to predicted price.

Risk of misuse: Predictions have $\pm\$25K$ uncertainty. Using as final price without human review would be irresponsible.

Recommendations: (1) Screening tool, not final decision. (2) Show $\pm\$25K$ range. (3) Flag predictions 30% off from neighbourhood avg. (4) Monitor errors per neighbourhood. (5) Retrain every 2–3 years.

Appendix A: Contribution Statement

This appendix describes the individual contributions of each team member to the capstone project, in compliance with the Academic Integrity section of the course guidelines.

Abdelmoghith El Asraoui

Abdelmoghith led the following areas:

- **Phase 1 Problem Framing & Data Acquisition:** Authored the problem statement and business motivation. Wrote the AI context note connecting the project to narrow AI concepts from the course. Developed the `load_data.py` ingestion script and performed the initial data audit. Set up the Git repository structure with README, `.gitignore`, and folder hierarchy.
- **Phase 2 Data Wrangling & EDA:** Conducted the full column audit (81 columns checked for types, nulls, and duplicates). Documented all cleaning decisions with technical rationale. Applied the IQR outlier detection method and generated 4 of the 8 visualizations (distribution plot, correlation heatmap, price vs. quality boxplot, and missing values heatmap).
- **Phase 3 Feature Engineering & Data Management:** Engineered 3 of the 5 new features (TotalSF, HouseAge, TotalBath). Wrote the SQLite storage pipeline and composed SQL Queries 1 and 3. Prepared the feature engineering log.

Mohamed Ben Mouh

Mohamed led the following areas:

- **Phase 2 Data Wrangling & EDA:** Applied the Z-score outlier detection method and compared results with the IQR approach, documenting the rationale for keeping all data. Implemented ordinal encoding for 9 quality features and one-hot encoding for all nominal features. Generated 4 of the 8 visualizations (price vs. living area, price vs. year built, neighborhood price distribution, and outlier boxplots). Wrote the EDA summary with the three key findings.
- **Phase 3 Feature Engineering & Data Management:** Engineered 2 of the 5 new features (TotalPorchSF, YearsSinceRemodel). Performed the correlation-based feature selection and authored SQL Queries 2 and 4 (custom: price by construction era).
- **Phase 4 Algorithm Selection:** Produced the 7-algorithm comparison table with strengths and weaknesses. Justified the selection of Linear Regression, Ridge, and XGBoost, and documented the rationale for rejecting SVR, Decision Tree, KNN, and Random Forest.
- **Phase 5 Model Training & Evaluation:** Implemented the Random Forest and

XGBoost models. Conducted GridSearchCV hyperparameter tuning for Ridge. Generated the residual plots and feature importance visualizations. Compiled the final model comparison table.

- **Phase 6 Model Card & Ethics:** Authored the model card covering intended use, training data, performance, limitations, and ethical considerations including fairness, bias, privacy, and misuse risk.

Joint Contributions

- Both members jointly reviewed, edited, and approved all deliverables.
- The final written report and slide deck were compiled collaboratively.
- All modeling code was cross-reviewed before execution.
- The conclusion and business recommendation were developed through joint discussion.

Bibliography

- [1] Kaggle, *House Prices — Advanced Regression Techniques*, 2016. <https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques>
- [2] De Cock, D., *Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project*, Journal of Statistics Education, Volume 19, Number 3, 2011.
- [3] Hoerl, A.E. & Kennard, R.W., *Ridge Regression: Biased Estimation for Nonorthogonal Problems*, Technometrics, Vol. 12, No. 1, 1970.
- [4] Chen, T. & Guestrin, C., *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD, 2016.
- [5] Pedregosa, F. et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, Vol. 12, 2011.
- [6] McKinney, W., *Data Structures for Statistical Computing in Python*, Proceedings of the 9th Python in Science Conference, 2010.
- [7] Mitchell, M. et al., *Model Cards for Model Reporting*, Proceedings of FAT*, 2019.